증명과 검증

halo2의 영지식 증명 시스템은 Setup, Prove, Verify 세 단계로 구성됩니다. 이 문서에서는 각 단계의 역할과 상호작용을 설명하고, 전체 증명 시스템의 동작 흐름을 다룹니다.

영지식 증명 시스템 개요

영지식 증명이란?

영지식 증명(Zero-Knowledge Proof)은 증명자(Prover)가 검증자(Verifier)에게 어떤 비밀 정보를 알고 있다는 것을 증명하면서도, 그 비밀 정보 자체는 공개하지 않는 암호학적 기법입니다.

halo2는 다음 세 가지 속성을 만족합니다:

1. **완전성(Completeness)**: 올바른 증명은 항상 검증됩니다
2. **건전성(Soundness)**: 거짓 증명은 검증되지 않습니다
3. **영지식성(Zero-Knowledge)**: 증명으로부터 비밀 정보를 알아낼 수 없습니다

halo2 증명 시스템의 구조

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
|     Setup    |──▶|     Prove    |──▶|   Verify   |
|              |     |              |     |            |
| • 회로 분석  |     | • Witness    |     | • 증명 검증 |
| • 키 생성    |     |     할당     |     | • 결과 반환 |
| • 매개변수   |     | • 증명 생성  |     |            |
|    설정      |     |              |     |            |
```

└──────────┘ └──────────┘ └──────────┘

Setup 단계

Setup의 역할

Setup 단계에서는 회로를 분석하고 증명 생성과 검증에 필요한 키와 매개변수를 생성합니다.

```
use halo2_proofs::{
plonk::{create_proof, keygen_pk, keygen_vk, verify_proof},
poly::kzg::{
commitment::{KZGCommitmentScheme, ParamsKZG},
multiopen::{ProverSHPLONK, VerifierSHPLONK},
strategy::SingleStrategy,
},
transcript::{Blake2bRead, Blake2bWrite, Challenge255},
};

// 1. 매개변수 생성 (Trusted Setup)
let k = 4; // 회로 크기 (2^k 행)
let params = ParamsKZG::<Bn256>::setup(k, OsRng);

// 2. 회로 인스턴스 생성
let empty_circuit = MyCircuit::default();
```

```
// 3. Verifying Key 생성
let  vk = keygen_vk(&params, &empty_circuit).expect("keygen_vk should not fail")


// 4. Proving Key 생성
let  pk = keygen_pk(&params, vk, &empty_circuit).expect("keygen_pk should not fa
```

Setup 과정의 세부 단계

1. 회로 분석

```
// 회로 분석 과정 (의사코드)
fn  analyze_circuit(circuit: &Circuit) -> CircuitAnalysis {
CircuitAnalysis {
num_advice_columns: circuit.advice_columns().len(),
num_instance_columns: circuit.instance_columns().len(),
num_fixed_columns: circuit.fixed_columns().len(),
num_gates: circuit.gates().len(),
num_lookups: circuit.lookups().len(),
degree: circuit.degree(),
}
}
```

2. 도메인 설정

```
// 계산 도메인 설정
let domain_size = 1 << k; // 2^k
let omega = Fr::root_of_unity(); // 단위근
let domain = Domain::new(domain_size, omega);
```

3. 고정 값 할당

```
// Fixed 컬럼의 값들을 미리 계산
fn assign_fixed_values(
circuit: &Circuit,
layouter: &mut impl Layouter<Fr>
) -> Result<(), Error> {
circuit.synthesize_fixed(layouter)?;
Ok(())
}
```

Trusted Setup vs Universal Setup

halo2는 **Universal Setup**을 사용합니다:

```
// Universal Setup - 한 번만 실행하면 모든 회로에 사용 가능
let universal_params = ParamsKZG::<Bn256>::setup(MAX_K, OsRng);

// 특정 회로용 키 생성
```

```
let circuit_specific_vk = keygen_vk(&universal_params, &circuit1)?;

let another_circuit_vk = keygen_vk(&universal_params, &circuit2)?;
```

Prove 단계

Prove의 역할

Prove 단계에서는 실제 witness 값들을 회로에 할당하고, 모든 제약조건이 만족됨을 보이는 증명을 생성합니다.

```
// 증명 생성 예제

fn generate_proof(

params: &ParamsKZG<Bn256>,

pk: &ProvingKey<G1Affine>,

circuit: MyCircuit,

instances: &[&[Fr]],

) -> Result<Vec<u8>, Error> {

let mut transcript = Blake2bWrite::<_, _, Challenge255<_>>::init(vec![]);

create_proof::<

KZGCommitmentScheme<Bn256>,

ProverSHPLONK<'_, Bn256>,

Challenge255<G1Affine>,

_,

Blake2bWrite<Vec<u8>, G1Affine, Challenge255<G1Affine>>,

_,
```

```
>(
params,
pk,
&[circuit],
&[instances],
OsRng,
&mut transcript,
)?;
Ok(transcript.finalize())
}
```

Prove 과정의 세부 단계

1. Witness 할당

```
// Witness 값들을 회로에 할당
impl Circuit<Fr> for MyCircuit {
fn synthesize(
&self,
config: Self::Config,
mut layouter: impl Layouter<Fr>
) -> Result<(), Error> {
layouter.assign_region(
|| "main computation",
```

```
|mut region| {
// 비밀 값들 할당
let a_cell = region.assign_advice(
|| "a",
config.advice[0],
0,
|| Value::known(self.a),
)?;
let b_cell = region.assign_advice(
|| "b",
config.advice[1],
0,
|| Value::known(self.b),
)?;
// 계산 결과 할당
let c_value = self.a + self.b;
let c_cell = region.assign_advice(
|| "c",
config.advice[2],
0,
|| Value::known(c_value),
)?;
Ok(())
```

```
},
)
}
}
```

2. 제약조건 검증

```
// 모든 제약조건이 만족되는지 확인 (의사코드)
fn verify_constraints_locally(
circuit: &Circuit,
witness: &Witness
) -> Result<(), ConstraintError> {
for constraint in circuit.constraints() {
let value = constraint.evaluate(&witness);
if value != Fr::zero() {
return Err(ConstraintError::Violation(constraint.id()));
}
}
Ok(())
}
```

3. Polynomial Commitment 생성

```
// 각 컬럼에 대한 다항식 commitment 생성 (의사코드)
```

```rust
fn create_commitments(
witness_polys: &[Polynomial<Fr>],
params: &ParamsKZG<Bn256>
) -> Vec<G1Affine> {
witness_polys
.iter()
.map(|poly| params.commit(poly))
.collect()
}
```

4. Challenge 생성

```rust
// Fiat-Shamir 변환을 통한 challenge 생성
let mut transcript = Blake2bWrite::init(vec![]);

// Commitment들을 transcript에 추가
for commitment in &commitments {
transcript.write_point(*commitment)?;
}

// Challenge 생성
let beta: Fr = transcript.squeeze_challenge_scalar::<()>();
```

```
let gamma: Fr = transcript.squeeze_challenge_scalar::<()>();
```

5. Opening Proof 생성

```
// 특정 점에서의 다항식 값과 증명 생성 (의사코드)
fn create_opening_proofs(
polys: &[Polynomial<Fr>],
challenges: &[Fr],
params: &ParamsKZG<Bn256>
) -> Vec<OpeningProof> {
polys
.iter()
.zip(challenges)
.map(|(poly, challenge)| {
let value = poly.evaluate(challenge);
let proof = params.open(poly, *challenge);
OpeningProof { value, proof }
})
.collect()
}
```

Verify 단계

Verify의 역할

Verify 단계에서는 증명자가 제공한 증명이 유효한지 검증합니다. 실제 witness 값을 알지 못해도 증명의 정확성을 확인할 수 있습니다.

```
// 증명 검증 예제
fn verify_proof(
params: &ParamsKZG<Bn256>,
vk: &VerifyingKey<G1Affine>,
proof: &[u8],
instances: &[&[Fr]],
) -> Result<(), Error> {
let strategy = SingleStrategy::new(params);
let mut transcript = Blake2bRead::<_, _, Challenge255<_>>::init(proof);
verify_proof::<
KZGCommitmentScheme<Bn256>,
VerifierSHPLONK<'_, Bn256>,
Challenge255<G1Affine>,
Blake2bRead<&[u8], G1Affine, Challenge255<G1Affine>>,
SingleStrategy<'_, Bn256>,
>(
params,
vk,
strategy,
&[instances],
&mut transcript,
```

```
)
}
```

Verify 과정의 세부 단계

1. 증명 파싱

```
// 증명에서 commitment들과 opening proof들 추출 (의사코드)
fn parse_proof(proof: &[u8]) -> ParsedProof {
let mut reader = ProofReader::new(proof);
ParsedProof {
advice_commitments: reader.read_commitments(num_advice_columns)?,
permutation_product_commitments: reader.read_commitments(num_permutations)?,
lookup_product_commitments: reader.read_commitments(num_lookups)?,
vanishing_h_commitments: reader.read_commitments(num_h_pieces)?,
opening_proofs: reader.read_opening_proofs()?,
}
}
```

2. Challenge 재생성

```
// 검증자가 동일한 challenge들을 재생성
fn regenerate_challenges(
commitments: &[G1Affine],
```

```
transcript: &mut impl Transcript<G1Affine, Fr>
) -> Challenges {
// Commitment들을 transcript에 추가
for commitment in commitments {
transcript.write_point(*commitment)?;
}
// 동일한 challenge들 생성
Challenges {
beta: transcript.squeeze_challenge_scalar::<()>(),
gamma: transcript.squeeze_challenge_scalar::<()>(),
alpha: transcript.squeeze_challenge_scalar::<()>(),
zeta: transcript.squeeze_challenge_scalar::<()>(),
}
}
```

3. Pairing 검증

```
// KZG commitment의 pairing 기반 검증 (의사코드)
fn verify_kzg_opening(
commitment: G1Affine,
point: Fr,
value: Fr,
proof: G1Affine,
params: &ParamsKZG<Bn256>
```

```
) -> bool {
// e(commitment - value*G, H) = e(proof, tau*H - point*H)
let lhs = pairing(
&(commitment - params.g1() * value),
&params.h()
);
let rhs = pairing(
&proof,
&(params.tau_h() - params.h() * point)
);
lhs == rhs
}
```

4. 제약조건 검증

```
// 제약조건들이 올바르게 만족되는지 검증 (의사코드)
fn verify_constraints(
commitments: &[G1Affine],
challenges: &Challenges,
vk: &VerifyingKey<G1Affine>
) -> bool {
// 각 gate의 제약조건 검증
for gate in &vk.cs.gates {
```

```
if !verify_gate_constraint(gate, commitments, challenges) {
return false;
}
}
// Permutation argument 검증
if !verify_permutation(commitments, challenges, &vk.permutation) {
return false;
}
// Lookup argument 검증
for lookup in &vk.cs.lookups {
if !verify_lookup(lookup, commitments, challenges) {
return false;
}
}
true
}
```

전체 워크플로우 예제

완전한 증명 시스템 구현

```
use halo2_proofs::{
circuit::{Layouter, SimpleFloorPlanner, Value},
```

```
plonk::*,
poly::kzg::commitment::ParamsKZG,
halo2curves::bn256::{Bn256, Fr, G1Affine},
};


// 1. 회로 정의
#[derive(Debug, Default, Clone)]
struct ExampleCircuit {
a: Value<Fr>,
b: Value<Fr>,
}


#[derive(Debug, Clone)]
struct ExampleConfig {
advice: [Column<Advice>; 3],
instance: Column<Instance>,
selector: Selector,
}


impl Circuit<Fr> for ExampleCircuit {
type Config = ExampleConfig;
type FloorPlanner = SimpleFloorPlanner;
```

```
fn  configure(meta: &mut ConstraintSystem<Fr>) -> Self::Config {
let  advice = [
meta.advice_column(),
meta.advice_column(),
meta.advice_column(),
];
let  instance = meta.instance_column();
let  selector = meta.selector();

meta.create_gate("add", |meta| {
let  s = meta.query_selector(selector);
let  a = meta.query_advice(advice[0], Rotation::cur());
let  b = meta.query_advice(advice[1], Rotation::cur());
let  c = meta.query_advice(advice[2], Rotation::cur());

vec![s * (a + b - c)]
});

ExampleConfig {
advice,
instance,
selector,
}
```

```
}

fn synthesize(
&self,
config: Self::Config,
mut layouter: impl Layouter<Fr>
) -> Result<(), Error> {
layouter.assign_region(
|| "add",
|mut region| {
config.selector.enable(&mut region, 0)?;

region.assign_advice(
|| "a",
config.advice[0],
0,
|| self.a,
)?;

region.assign_advice(
|| "b",
config.advice[1],
0,
```

```
|| self.b,
)?;

region.assign_advice(
|| "c",
config.advice[2],
0,
|| self.a + self.b,
)?;

Ok(())
},
)
}
}

// 2. 전체 워크플로우
fn full_proof_workflow() -> Result<(), Box<dyn std::error::Error>> {
// Setup
let k = 4;
let params = ParamsKZG::<Bn256>::setup(k, OsRng);
let empty_circuit = ExampleCircuit::default();
let vk = keygen_vk(&params, &empty_circuit)?;
```

```
let pk = keygen_pk(&params, vk, &empty_circuit)?;


// Prove
let circuit = ExampleCircuit {
a: Value::known(Fr::from(2)),
b: Value::known(Fr::from(3)),
};
let public_input = vec![Fr::from(5)]; // a + b = 5
let instances = vec![public_input.as_slice()];
let mut transcript = Blake2bWrite::<_, _, Challenge255<_>>::init(vec![]);
create_proof::<
KZGCommitmentScheme<Bn256>,
ProverSHPLONK<'_, Bn256>,
Challenge255<G1Affine>,
_,
Blake2bWrite<Vec<u8>, G1Affine, Challenge255<G1Affine>>,
_,
>(
&params,
&pk,
&[circuit],
&[instances.as_slice()],
OsRng,
```

```
&mut  transcript,
)?;
let  proof = transcript.finalize();


// Verify
let  strategy = SingleStrategy::new(&params);
let  mut  transcript = Blake2bRead::<_, _, Challenge255<_>>::init(&proof[..]);
verify_proof::<
KZGCommitmentScheme<Bn256>,
VerifierSHPLONK<'_, Bn256>,
Challenge255<G1Affine>,
Blake2bRead<&[u8], G1Affine, Challenge255<G1Affine>>,
SingleStrategy<'_, Bn256>,
>(
&params,
pk.get_vk(),
strategy,
&[instances.as_slice()],
&mut  transcript,
)?;


println!("증명이 성공적으로 검증되었습니다!");
Ok(())
```

```
}
```

성능 고려사항

1. Setup 최적화

```
// Setup 결과 캐싱
use std::collections::HashMap;

struct SetupCache {
params_cache: HashMap<u32, ParamsKZG<Bn256>>,
vk_cache: HashMap<String, VerifyingKey<G1Affine>>,
}

impl SetupCache {
fn  get_or_create_params(&mut  self, k: u32) -> &ParamsKZG<Bn256> {
self.params_cache.entry(k).or_insert_with(|| {
ParamsKZG::<Bn256>::setup(k, OsRng)
})
}
}
```

2. 증명 생성 최적화

```rust
// 병렬 처리를 통한 성능 향상
use rayon::prelude::*;

fn parallel_witness_assignment(
regions: &mut [Region],
witnesses: &[WitnessData]
) -> Result<(), Error> {
regions
.par_iter_mut()
.zip(witnesses.par_iter())
.try_for_each(|(region, witness)| {
assign_witness_to_region(region, witness)
})
}
```

3. 검증 최적화

```rust
// 배치 검증으로 여러 증명을 효율적으로 검증
fn batch_verify_proofs(
params: &ParamsKZG<Bn256>,
vk: &VerifyingKey<G1Affine>,
proofs: &[ProofData]
) -> Result<bool, Error> {
```

```
// 모든 증명을 한 번에 검증
let strategy = AccumulatorStrategy::new(params);
for proof_data in proofs {
verify_single_proof_in_batch(vk, proof_data, &strategy)?;
}
strategy.finalize()
}
```

오류 처리 및 디버깅

일반적인 오류들

```
// 증명 생성 실패 원인들
#[derive(Debug)]
enum ProofError {
ConstraintViolation { gate_index: usize, row: usize },
WitnessAssignmentError { column: usize, row: usize },
SetupParameterMismatch,
InsufficientRandomness,
}

// 디버깅 도구
fn debug_proof_generation(circuit: &Circuit) {
```

```
// MockProver를 사용한 로컬 검증
let prover = MockProver::run(k, circuit, instances).unwrap();
if let Err(errors) = prover.verify() {
for error in errors {
println!("검증 오류: {:?}", error);
}
}
}
```

다음 단계

증명과 검증의 기본 개념을 이해했다면:

- Circuit 설계: 실제 회로 구현 방법 학습
- Proof 생성과 검증: 더 자세한 증명 시스템 구현
- 실습 예제: 실제 예제를 통한 실습

참고 자료

- PLONK 논문 - 증명 시스템의 이론적 배경
- KZG Commitment 논문
- halo2 증명 생성 API